

заголовки Спецификация языка программирования Dart

Сфера применения

Стандарт Еста определяет синтаксис и семантику Язык программирования Dart. В нем не указаны API библиотек Dart. За исключением тех случаев, когда библиотечные элементы необходимы для Правильное функционирование самого языка (например, существование класса) с помощью методов , .

Соответствие.

Соответствующая реализация языка программирования Dart должен обеспечивать и поддерживать все API; (библиотеки, типы, функции, геттеры, сеттеры, будь то верхнего уровня, статический, экземпляр или локальный предусмотренных в данной спецификации.

Допускается соответствующая реализация для предоставления дополнительных API, но не дополнительный синтаксис, За исключением экспериментальных особенностей.

Нормативные ссылки

Следующие документы являются незаменимыми для применения данного документа. Для датированных ссылок применяется только цитируемое издание. Для недатированных ссылок последнее издание ссылающегося документа Применяются (включая любые поправки).

Стандарт Unicode версии 5.0 с изменениями, внесенными Unicode 5.1.0 или его преемником.

Ссылка на API Dart, <https://api.dartlang.org/>

Термины и определения

Термины и определения, используемые в данной спецификации, приведены в корпус собственно спецификации.

Примечание)

Различают нормативный и ненормативный тексты. Нормативный текст определяет правила Дарта. Он приведен в этом шрифте. В настоящее время ненормативный текст включает:

[Обоснование] Обсуждение мотивации языковых дизайнерских решений происходит курсивом. Отличие нормативного от ненормативного помогает уточнить какая часть текста является обязательной и какая часть является просто разъяснительной. ? [Комментарий] Комментарии такие как Внимательный читатель заметит Имя Дарт имеет четыре символа ? ” служит для иллюстрации или уточнения спецификации, Они не соответствуют нормативному тексту. <Разница между комментарием и обоснованием может быть тонкой. ? Комментарий более общий, чем обоснование. могут включать иллюстративные примеры или пояснения.?

Зарезервированные слова и встроенные идентификаторы (перенаправлено с «R000372») » жирный шрифт.

Примерами могут служить **switch** или .?

Грамматические постановки приведены в общем варианте EBNF. Левая сторона производства заканчивается на ::=. С правой стороны чередование представлено вертикальными полосами, и секвенирование

по интервалу. Как и в ПЭГ, чередование отдает приоритет левым. Факультативные элементы производства закрепляются вопросительным знаком. Например: . Присоединение звезды к элементу производственного средства. Он может повторяться ноль или более раз. Добавление знака плюс к произведению означает, что это происходит один или несколько раз. Парантезы используются для группирования. Отрицание представлено префиксом элемента производства с тильдой. Отрицание подобно некомбинатору ПЭГ, Но он потребляет вход, если он совпадает. В контексте лексического производства он потребляет один персонаж, если он есть; В противном случае, один токен, если он есть.

Примером может служить:?

```

<aProduction> ::= <anAlternative>
| <anotherAlternative>
| <oneThing> <after> <another>
| <zeroOrMoreThings>*
| <oneOrMoreThings>+
| <anOptionalThing>?
| (<some> <grouped> <things>)
| ~<notAThing>
| 'aTerminal'
| <A_LEXICAL_THING>

```

Таким образом представлены как синтаксические, так и лексические произведения. Лексические постановки отличаются своими названиями. Названия лексических произведений состоят исключительно из Верхние символы и подчеркивания. Как всегда, в грамматических произведениях Белое пространство и комментарии между элементами производства неявно игнорируются, если не указано иное. Токены препинания появляются в кавычках.

Производственные мощности, насколько это возможно, При обсуждении представленных ими конструкций.

P000434 является синтаксической конструкцией. Это может считаться частью текста, который можно вывести в грамматике. Это может быть дерево, созданное таким образом. для данного термина Это синтаксическая конструкция которое соответствует непосредственному поддереву рассматривается как производное дерево. данного термина является, или немедленный субтермин, или субтермин немедленного субтермина .

Список $x_1, \dots, x_n$ обозначает любой список «<p000010> Элементы формы. Обратите внимание, что Может быть нулевой, в этом случае список пуст. Мы широко используем такие списки в этой спецификации.

Для, Пусть будет атомным синтаксическим объектом (как идентификатор), составная синтаксическая сущность (например, выражение или тип); и E снова составная синтаксическая сущность. Запись $[x_1/y_1, \dots, x_n/y_n]E@[x/y \dots]E$ Затем обозначает копию В котором каждое появление Он был заменен на «P000020».

Эта операция также известна как R000437. Это тот вариант, который позволяет избежать захвата. То есть, когда Содержит конструкцию, которая вводит в вложенный объем для некоторых, Замена не заменяет в этой сфере. И наоборот, если такая замена поставит идентификатор (перенаправлено с «R000025») где объявляется, Соответствующие декларации в Они систематически переименовываются в новые имена.

Короче говоря, свобода захвата гарантирует, что «смысл» каждого идентификатора Сохраняется при замене.?

Иногда мы злоупотребляем списком или буквальным синтаксисом карты, написав $o_1, \dots, o_n$ (соответственно $;, k_n; o_n$) где и k_i . Это могут быть объекты, а не выражения. Цель состоит в том, чтобы обозначить список (соответственно карту) объекта. Комментарии к статье (соответственно, ключами которого являются) и значения являются .

Спецификации операторов часто включают такие заявления, как: »ор эквивалентно методу вызова . $or()$ $x.or(y)@$. $or()$. Такие спецификации следует понимать как сокращение для:

эквивалентно методу вызова . $or'()$, Предполагая класс фактически объявлен неоператорский метод, названный Определение функции оператора .

<Это обрезание необходимо потому, что . $or()$, где or является оператором, Это не юридический синтаксис. Тем не менее, это болезненно многословно, и мы предпочитаем изложить это правило здесь, Используйте краткую и четкую запись в спецификации.?

Когда спецификация относится к порядку, заданному в программе, означает порядок исходного текста программы, Сканирование слева направо и сверху вниз.

Когда спецификация относится к свежие переменныеизменяемые! свежий. означает локальную переменную с именем, которое нигде не встречается. в текущей программе. Когда спецификация вводит новую переменную, связанную с объектом, Свежая переменная неявно связана с окружающей областью.

Ссылки на иные неуказанные названия субъектов программы (например, классы или функции) Они интерпретируются как имена членов основной библиотеки Дарта.

Примерами могут служить классы и Представляя, соответственно, корень классовой иерархии и Реификация типов Runtime. Можно было бы заявить, например, локальная переменная, называемая , Поэтому, как правило, неверно предполагать, что Имя На самом деле, мы решим, что это основной класс. Однако мы обычно не упоминаем об этом, для краткости.?

Когда в спецификации указано, что одна часть синтаксиса другой фрагмент синтаксиса, это означает, что он эквивалентен во всех отношениях; и прежний синтаксис должен генерировать те же ошибки компиляции и имеют такое же поведение во время выполнения, как и последние, если таковые имеются. Сообщения об ошибках, если таковые имеются, всегда должны относиться к исходному синтаксису. ? Если считается,

что выполнение или оценка конструкции эквивалентно исполнению или оценке другой конструкции, Тогда только время выполнения эквивалентно, Ошибки времени компиляции применяются только к исходному синтаксису.

Когда в спецификации указано, что одна часть синтаксиса это
Еще одна часть синтаксиса, Это означает, что статический анализ
Статический анализ (в частности, происходят одни и те же ошибки компиляции). Более того, если не имеет ошибок времени компиляции
Поведение Во время выполнения точно такое же поведение.

Ошибка сообщения, если таковые имеются, Всегда следует обращаться к исходному синтаксису. ?

Короче говоря, всякий раз, когда Обозначается как, читатель должен немедленно перейти к разделу о Для получения дополнительной информации о Статический анализ и динамическая семантика. ?

<Понятие «обработка» аналогично понятию синтаксического сахара:
Обозначается как ” Также можно было бы написать Он был преобразован в «R000067». Конечно, тогда его действительно следует назвать «семантический сахар». в силу применимости преобразования и построения Может опираться на информацию статического анализа.

Дело в том, что мы конкретизируем только статический анализ и динамическую семантику. Основной язык, который является подмножеством Dart (немного меньше, чем Dart) Преобразование любой программы Дарта в Программа на этом базовом языке. Это помогает поддерживать языковую спецификацию последовательной и понятной. Потому что он показывает прямо что некоторые особенности языка вводят существенную семантику, и другие лучше описывать как простые сокращения существующих конструкций. ?

Спецификация использует одну синтаксическую конструкцию,
) let expression@let выражение который не может быть выведен в грамматике (то есть ни один исходный код Dart не содержит такого выражения). Это выражение полезно для определения определенных синтаксических форм. которые рассматриваются как другие синтаксические формы, поскольку она позволяет вводить и инициализировать одну или несколько свежих переменных, и использовать их в выражениях.

То есть, Выражение вводится только как инструмент. Определение оценочной семантики выражения в терминах других выражений, содержащих выражения.?

Синтаксис Выражение выглядит следующим образом:

<letExpression> ::= **let** <staticFinalDeclarationList> <выражение> e_{let}, e_j, v_j, k

Пусть будет **let**Выражение формы. Негласно предполагается, что является свежей переменной, , Если только не указано обратное.

Содержит вложенные рамки, S1k Сфера охвата для является текущим объемом для, и прилагаемая область для является, . Текущий объем является текущим объемом, Текущий объем составляет, , и текущий объем является. Для, вводит конечную локальную переменную в, Статический тип как заявленный тип.

Типовой вывод и тип контекста, используемый для вывода не являются актуальными. Обычно предполагается, что вывод типа уже произошел. ()?

Оценка доходы от оценка » к объекту » и связывание с , где , в указанном порядке. Наконец, оценивается на объект А потом Показатель

Правая граница каждой страницы в этом документе используется для указания Ссылки на объекты.

Документ содержит индекс в конце. Каждая запись в индексе относится к номеру страницы . На странице На марже есть «♦» при определении данной индексируемой фразы, и сама фраза показана с использованием этот шрифт. Настоящим мы представляем себя.

Правый край также содержит символы. Всякий раз, когда символ (скажем, или) вводится и используется более чем в нескольких строках x_j текста; Это показано на полях.

Дело в том, что легко найти определение символа. Сканирование текста до тех пор, пока этот символ не появится на полях. Чтобы избежать бесполезной многословности, некоторые символы не упоминаются на полях. Например, мы можем ввести e1k, Но показывают только и на полях.

Обратите внимание, что может потребоваться посмотреть несколько строк текста. Надпись «P000115» или символ, Потому что маргинальные маркеры могут быть сдвинуты вниз по одной линии. когда имеется более одного маркера для одной строки. ?

Обзор

Дарт — это классовое, однонаследное, чистоеОбъектно-ориентированный язык программирования. Dart необязательно типизирован () и поддерживает модифицированные дженерики. Тип времени выполнения каждого объекта представлен как Пример класса который может быть получен Позвонив Геттеру в классе , Корень иерархии классов Дарта.

Программы Дарта можно статически проверять. Программы с ошибками времени компиляции не имеют заданной динамической семантики. Данная спецификация не дает возможности ответить на дополнительные вопросы. О библиотеке или программе в данный момент где, как известно, имеется ошибка времени компиляции.

Однако инструменты могут поддерживать выполнение некоторых программ с ошибками. Например, компилятор может компилировать определенные конструкции с ошибками, такими, что Динамическая ошибка возникает при попытке осуществить такую конструкцию, или интегрированное время выполнения IDE может поддерживать открытие окно редактора при выполнении такой конструкции, Это позволяет разработчикам исправить ошибку. Предполагается, что такие признаки будут представлять собой естественное расширение Динамическая семантика Дарта, как указано здесь, но, как уже упоминалось, Эта спецификация не пытается точно указать, что это означает. ?

Как указано в этом документе, динамические проверки гарантированно выполняются в определенных ситуациях; и некоторые нарушения системы типа бросают исключения во время выполнения.

Реализация может опускать такие проверки всякий раз, когда они гарантированный успех, например, на основе результатов статического анализа. ?

Существование между факультативной типизацией и реификацией основывается на следующем:

Информация Reified Type отражает типы объектов во время выполнения и всегда могут быть запрошены динамическими конструкциями проверки типа (аналоги `exampleOf`, `casts`, `typecase` и т. д. на других языках). Информация о типе включает в себя Доступ к экземплярам класса представлять типы, тип времени выполнения (класс `aka`) объекта, и фактические значения параметров типа Конструкторы и общие вызовы функций.

Типовые аннотации декларируют типы переменные и функции (включая методы и конструкторы);

Аннотации типа могут быть опущены, и в этом случае они обычно Заполняется шрифтом **dynamic** (P000375). ?

Реализованный Dart включает в себя обширную поддержку вывода опущенных типов. Эта спецификация делает предположение, что вывод имел место, Следовательно, предполагаемые типы уже присутствуют в программе. Однако в некоторых случаях информация отсутствует. чтобы сделать вывод о пропущенном типе аннотации, И поэтому эта спецификация все еще должна указывать, как с этим бороться. Будущая версия этой спецификации также будет указывать тип вывода. ?

Программы Dart организованы по модульному принципу. единицы, называемые библиотеки (P000376). Библиотеки являются единицами инкапсуляции и могут быть взаимно рекурсивными.

Но это не первый класс. Чтобы получить несколько копий библиотеки, работающей одновременно, Для этого необходимо создать изолятору.?

Выполнение программы dart может происходить с включенными или отключенными утверждениями. Метод, используемый для включения или отключения утверждений, является специфическим для реализации.

0.1 Скопинг

А пространство имён времени компиляции `namespace!compiletime` Это частичная функция, которая отображает имена для значений пространства имен. Пространства имён времени компиляции используются гораздо чаще, чем пространства имён времени выполнения (определено ниже в этом разделе), Когда слово используется в одиночку, Это означает пространство имён компиляции времени. является лексическим токеном, который является ан Идентификатор, за которым следует

=, или ан оператор, ; это Декларация, пространство имен или специальное значение (P000377).

Если Тогда мы говорим, что `mapnamespace!maps a key to a value` тот `keynamespace!key`

`valuenamespace!value` , и это имеет связывание `namespace!has a binding` . Дело в том, что Частичные функции означают, что Каждое имя отображается как минимум до одного значения пространства имен. То есть, если имеет привязки и ?

Пусть будет пространством имен. Мы говорим, что имя Если является ключом от . Мы говорим декларацию имеется в виду) Если ключ Соответствует .

Область применения имеет связанное пространство имен 0. Обязательства 0 указаны в этом документе тем, что Данная декларация Название: P000129 `introducesdeclaration!intro`тро предприятие в сферу применения. конкретный субъект V в , что означает связывание Добавлено в 0.

В некоторых случаях название декларации отличается от идентификатор, который встречается в синтаксисе декларации, используемом для его объявления. Сеттеры имеют имена, отличные от соответствующих геттеров. потому что у них всегда есть автоматически добавляется в конце, и унарный оператор минус имеет специальное название .?

Как правило, Декларация себя, Но есть исключения. Например, переменная декларация вводит неявно индуцированную декларацию Геттера; а в некоторых случаях и неявно индуцированной установочной декларацией в Сфера применения.?

Обратите внимание, что метки () не включены в пространство имен области. Они решаются лексически, а затем просматриваются в пространстве имен. ?

Это ошибка времени компиляции, если существует более одного объекта с тем же именем. заявленных в том же объеме.

Поэтому невозможно, например, определить класс, который заявляет, что Метод и геттер с тем же названием в Дарте. Точно так же нельзя объявить функцию верхнего уровня Название библиотеки переменная или класс Объявлено в той же библиотеке.?

Мы вводим понятие а . Это частичная функция от имен до сущностей времени выполнения, В частности, места хранения и функции. Каждое пространство имен во время выполнения соответствует пространству имен с одинаковыми ключами. но со значениями, соответствующими семантике значений пространства имен.

<Пространство имен обычно отображает имя в декларацию, И его можно использовать статически, чтобы выяснить, что это имя означает. Например, переменная связана с фактическим местом хранения во время выполнения. Мы вводим понятие пространства имен во время выполнения, основанного на пространстве имен, так, чтобы динамическая семантика могла получить доступ к объектам времени выполнения Как это место хранения. Один и тот же код может выполняться несколько раз с одним и тем же пространством

имен во время выполнения. или с различными пространствами имен времени выполнения для каждого исполнения. Например, локальные переменные, объявленные внутри функции. специфичны для каждого вызова функции, и переменные экземпляра специфичны для объекта.?

Дарт лексически ограничен. Сферы могут гнездиться. Имя или декларация Если находится в пространстве имен, индуцируемом или Если доступен в лексически закрытом диапазоне . Мы говорим, что имя или декларация является Если доступен в текущем объеме.

Если заявление Оригинальное название: P000143 находится в пространстве имен, индуцируемом областью , Любое заявление под названием Это доступно в лексически замкнутой области .

Следствием этих правил является то, что можно скрыть тип Метод или переменная. Соглашения об именах обычно предотвращают такие злоупотребления. Тем не менее, следующие программы являются законными: ?

Наименования могут вводиться в сферу действия посредством заявлений в пределах сферы действия. или другими механизмами, такими как импорт или наследование.

Взаимодействие лексики и наследования является тонким. В конечном счете, вопрос заключается в том, является ли лексический анализ преобладает над наследством или наоборот. Дарт выбирает первое.

Позволить наследственным именам иметь приоритет над местными именами Они могут создавать неожиданные ситуации по мере развития кода. В частности, поведение кода в подклассе может незаметно измениться. Если в суперклассе вводится новое имя. Рассмотрим:?

Теперь предположим метод Добавлено в .?

Если наследование превалирует над лексическим размахом, Поведение Изменится неожиданным образом. Ни один из авторов Автор статьи Они обязательно осознают это. В Дарте, если существует лексически видимый метод , Это всегда будет называться.

Теперь рассмотрим противоположный сценарий. Начнем с версии который содержит , Но не объявляйте В библиотеке . Опять же, есть изменения в поведении, но автор Это тот, кто ввел несоответствие, которое влияет на их код. Новый код является лексически видимым. Оба эти фактора повышают вероятность обнаружения проблемы.

Эти соображения становятся еще более важными. Если ввести такие конструкции, как вложенные классы, которые могут быть рассмотрены в будущих версиях языка.

Хорошие инструменты должны, конечно, стремиться информировать программистов. В таких ситуациях (с осторожностью). Например, идентификатор, который является как наследственным, так и лексически видимым. Можно выделить (через подчеркивание или окраску). Более того, тесная интеграция контроля источника с инструментами языкового знания Выявляет такие изменения, когда они происходят.?

Конфиденциальность

Dart поддерживает два уровня P000449: публичной и частной. Декларация `privateprivate!declaration` Если его имя является частным, В противном случае это `publicpublic!declaration`. Имя `private f` любой `Δ` из идентификаторов, содержащих является частным, В противном случае это `publicpublic!name`. Идентификатор: `privateprivate!identifier` если оно начинается с подчеркивания (`_`) характер В противном случае это `publicpublic!identifier`.

Декларация является *accessible to a library* Если Объявляется в Если `Δ` является публичным.

<Это означает, что частные декларации могут быть доступны только в пределах Библиотека, в которой они объявлены.?

Конфиденциальность распространяется только на декларации в библиотеке. Не в библиотечную декларацию в целом. Это потому, что библиотеки не ссылаются друг на друга по имени. Идея частной библиотеки бессмысленна. (P000379). Если название библиотеки начинается с символа, не влияет на доступность библиотеки или ее членов. ?

Конфиденциальность на данный момент является статическим понятием, связанным с определенным фрагмент кода (библиотека). Он предназначен для поддержки проблем разработки программного обеспечения Вместо проблем безопасности. Ненадежный код всегда должен работать в другом Изоляторе.

Конфиденциальность указывается именем декларации - отсюда Приватность и имена не являются ортогональными. Это имеет то преимущество, что и люди, и машины может признавать доступ к частным декларациям в точке использования; без знания контекста, из которого выводится декларация. ?

Конкуренция

Код Дарта всегда однопоточный. В Дарте нет параллели между общими государствами. Конкурентность поддерживается через актероподобные сущности, называемые .

Изолировать Термин — это единица параллелизма. У него есть собственная память и собственная нить управления. изолят осуществляет связь посредством передачи сообщений (`()`). Ни одно государство никогда не делится между собой. Изолятор создается путем нереста (`()`).

Ошибки и предупреждения

Эта спецификация различает несколько видов ошибок.

Ошибки времени компиляции Ошибки, которые исключают исполнение. Ошибка времени компиляции должна сообщаться компилятором Dart. До того, как будет выполнен ошибочный код.

«Реализация Дарта имеет значительную свободу. Когда происходит компиляция. Современные реализации языков программирования часто переплетаются компиляция и исполнение, так, чтобы компиляция способа могла быть отложена, например, до тех пор, пока он не будет впервые использован. Следовательно, ошибки времени компиляции в методе может быть сообщено Это произошло во время первого вызова P000156.

Дарт часто загружается непосредственно из источника. без промежуточного бинарного представления. В интересах быстрой загрузки реализация Dart Например, можно избежать полного разбора методических тел. Это можно сделать путем токенизации ввода и проверка сбалансированных кудрявых брекетов при входе в тело. В такой реализации будут обнаружены даже синтаксические ошибки только в том случае, если метод необходимо выполнить, В какое время он будет собран (смеется).

В среде разработки компилятор должен сообщать об ошибках компиляции, чтобы наилучшим образом служить программисту.

Среда развития Дарта может выбрать поддержку устранения ошибок при преобразовании программ, например, Замена ошибочного выражения вызовом отладчика. выходит за рамки настоящего документа Чтобы определить, как работают такие преобразования, где они могут быть использованы.?

Если происходит непойманная ошибка времени компиляции в коде запущенного изолированного термина , немедленно приостанавливается. Единственным обстоятельством, при котором может быть обнаружена ошибка компиляции времени, является С помощью кода работает отражательно, где зеркальная система может поймать его.

«Как правило, после ошибки компиляции и приостановлено, будет прекращен. Однако это зависит от общей окружающей среды. Двигатель Dart работает в контексте , Программа, которая взаимодействует между двигателем и окружающей вычислительной среды. Время выполнения может быть, например, программой C++ на сервере. Когда изолятор терпит неудачу с ошибкой времени компиляции, как описано выше, контроль возвращается во время выполнения, За исключением случая, описывающего проблему. Это необходимо для того, чтобы время выполнения могло очищать ресурсы и т.д. Именно тогда время выполнения ршеает, прекращать ли изоляторе или нет.?

Статические предупреждения ситуации, которые не препятствуют исполнению, которые, однако, вряд ли будут Они могут вызвать проблемы или неудобства. А. Компилятор Dart может сообщать обо всех, или ни одно из указанных статических предупреждений до выполнения соответствующего кода. А. Компилятор Dart также может сообщать о дополнительных предупреждениях. не определены данной спецификацией.

Когда эта спецификация говорит, что происходит, Это означает, что соответствующий объект ошибки выбрасывается. Когда говорится, что происходит, представляет собой неудавшуюся проверку типа во время выполнения, и предмет, который брошен орудиями .

Всякий раз, когда мы говорим, что исключение это *thrown*, Он действует так, как будто его бросило выражение (). как объект исключения и со следом стека соответствующее текущему состоянию системы. Когда мы говорим, что выбрасываетсябросает класс, где Это класс, мы имеем в виду, что бросается экземпляр класса .

Если непойманное исключение выкинуто бегущим изолят , немедленно приостанавливается.

Переменные.

Переменные — это места хранения в памяти.

```
<finalConstVarOrType>:= <тип> <тип> <varOrType>
<varOrType> ::= < <type>
<initializedVariableDeclaration> ::= < <declaredIdentifier> ('=' <expres-
sion>)? (';' <initialized> Идентификатор) *
<initializedIdentifier> ::= <identifier> ('=' <expression>)?
<initializedIdentifierList> ::= <initializedIdentifier> (';' <initialized> Идентификатор)
*
```

инициализированная переменная декларация Объясняет две или более переменных эквивалентны множественным переменным декларациям, объявляющим одинаковый набор переменных имен, в том же порядке, с той же инициализацией, типом и модификатором.

Например, `x, y;` является эквивалентным `x; y;` и `static late finalString s1, s2 = "foo";` эквивалентно тому, чтобы иметь оба `static late finalString s1;` и `static late finalString s2 = "foo";`?

Можно включить в переменную декларацию модификатор. Эффект от выполнения этого с переменной экземпляра описан в другом месте. (P000383). Это ошибка времени компиляции для объявления переменной. которая не является переменной экземпляра для включения модификатора

Формальное объявление параметров индуцирует локальную переменную в область. Но формальные декларации о параметрах не являются вариативными. Не допускайте вышеописанных ошибок. Влияние наличия модификатора на формальный параметр описывается в другом месте ()?

В переменной декларации одной из форм или $N \text{ id} = e$; где N происходит от $\langle metadata \rangle$ $\langle finalConstVarOrType \rangle$ и id является идентификатором, Мы говорим, что является идентификатора. Для каждого идентификатора, который не является декларирующим событием, Мы говорим, что это . Мы также сокращаем это, чтобы сказать, что идентификатор а соответственно .

В выражении формы Возможно, что имеет статический тип **dynamic** и не могут быть связаны с любое конкретное заявление под названием во время составления,Но в этой ситуации Идентификатор ссылки.?

Для краткости мы будем ссылаться на переменную, используя ее название. Даже если имя переменной и сама переменная Это очень разные понятия.

Итак, мы поговорим о переменной . Вместо введения переменной Оригинальное название: P000570 , Для того, чтобы потом можно было сказать "переменная". Это не должно создавать двусмысленности. поскольку понятие имени и понятие переменной Они очень разные.?

является переменной декларацией, идентификатор декларирования которой Далее следуют и .

[illegible]

ошибка времени компиляции возникает, если статическая или библиотечная переменная имеет отсутствие инициализации выражения и тип, который не является нулевым (). Если в переменной декларации нет модификатор `static` или модификатор `extern`.

А постоянная переменная **вариабельная!** постоянная переменная, в которую включается модификатор `const`. Постоянная переменная должна быть инициализирована в постоянное выражение. `0`, ошибка времени компиляции.

Это грамматически невозможно для постоянной переменной декларации. Модификатор `const`. Даже если бы это было грамматически возможно, а Постоянная переменная все равно должна быть ошибкой времени компиляции. Потому что быть постоянной времени компиляции по своей сути несовместимо. быть просчитанным поздно.

Неявно индуцированные геттеры и сеттеры
Произошел

Геттер: Переменная с объявленным типом Рассмотрим переменную
декларацию одной из форм

где Тип: является идентификатором, и ? указывает, что данный модификатор может присутствовать или отсутствовать. Каждая из этих деклараций неявно вызывает геттер. () с заголовком, чье обращение оценивается как описано ниже (P000391). В этих случаях заявленный тип является .

12

```
var id=e;  
final id=e; const id=e;
```

«<P001010> неявно вызывает геттер с заголовком, который Содержит **static** если f декларация содержит **static** После чего следует , где Получено из вывода типа в случае, когда существует, и является в противном случае. Призыв этого геттера оценивает, как описано ниже. (P000392). В этих случаях заявленный тип является .

Сеттер: Переменная переменная с объявленным типом Переменная декларация одной из форм

```
T id=e;
```

неявно вызывает сеттер () с заголовком , чье исполнение устанавливает значение для входящего аргумента .

Сеттер: Переменная переменная без объявленного типа с инициализацией Переменная декларация формы **var id=e;** неявно вызывает сеттер с заголовком , чье исполнение устанавливает значение для входящего аргумента .

Вывод типа мог бы обеспечить тип, отличный от ()?

Сеттер: изменчивая переменная без объявленного типа, без инициализации Переменная декларация формы

неявно вызывает сеттер с заголовком , чье исполнение устанавливает значение для входящего аргумента .

Тип вывода еще не определен в этом документе (). Обратите внимание, что тип вывода может меняться, например, , Что приведет нас к более раннему случаю.?

Сеттер: Поздно-финальная переменная с объявленным типом Переменная декларация формы

неявно вызывает сеттер () с заголовком . Если этот сеттер выполнен в ситуации, когда переменная не был связан, Он связывает с объектом, который Это неизбежно. Если этот сеттер выполнен в ситуации, когда переменная был привязан к объекту, Возникает динамическая ошибка.

Переменная декларация формы

неявно вызывает сеттер с заголовком . Казнь указанного сеттера в ситуации, когда переменная не был связан связываться к объекту, что аргумент Это неизбежно. Исполнитель The Setter в ситуации, когда переменная был привязан к объекту Это приведет к динамической ошибке.

Сфера, в которую вводятся неявные геттеры и сеттеры Это зависит от вида варьируемой декларации.

Переменная библиотеки вводит геттер в библиотечный объем ограждающей библиотеки. Статическая переменная вводит статический геттер в сфера действия корпуса немедленно закрывающего класс, микширование, enum или расширение декларации. Переменная экземпляра вводит в сфера действия корпуса немедленно закрывающего класс, микширование или декларирование enum (расширение не может декларировать переменные экземпляра).

Нелокальная переменная вводит сеттер, если `f` не имеет модификатора **final** или модификатора **const**, или `i`, но не имеет инициализирующего выражения.

Переменная библиотеки, которая вводит сеттер, введет библиотечный сеттер в объем библиотеки. Статическая переменная, которая вводит сеттер, введет статический сеттер в область тела немедленно закрывающего класс, микширование, `enum` или расширение декларации. Переменная экземпляра, которая вводит установщик, введет инстанция, установленная в объем тела немедленно закрывающего класс, микширование или декларирование `enum` (расширение не может декларировать переменные экземпляра).

Быть переменной, объявленной переменной декларацией имеет начальное выражение. Это ошибка времени компиляции, если статический тип не присваивается заявленному типу. Это ошибка времени компиляции, если переменная конечного экземпляра заявление которого имеет инициализаторное выражение. Он также инициализируется конструктором, либо путем инициализации формального или инициализатора списка.

Ошибка времени компиляции, если переменная последнего экземпляра который был инициализирован посредством форма инициализации конструктора. Также инициализируется в списке инициализаторов (P000397).

Декларация непоздней статической переменной Оригинальное название: P000591 имеет модификатор **final** или модификатор **const** не вызывает сеттера. Однако назначение на в месте, где находится в сфере. Это не обязательно ошибка времени компиляции. Например, сеттер по имени. Можно найти по лексическому поиску (P000398).

Аналогично, переменная непоздней конечной инстанции не вызывает сеттера, Но назначение может быть призывом к Сеттер, который предоставляется другим способом. Например, может быть, что лексический поиск ничего не дает. и место назначения имеет доступ к , Интерфейс ограждающего класса имеет сеттер под названием (в этом случае и геттер, и сеттер наследуются)?

Рассмотрим переменную Декларация которого не имеет модификатора. Предположим, что не имеет начального выражения, И это не инициализируется инициализацией формального (), ни элементом в списке инициализаторов конструктора (перенаправлено с «P000400»). Начальное значение Тогда нулевой объект ().

Обратите внимание, что существует множество ситуаций, когда такое заявление переменной Ошибка времени компиляции, В этом случае начальное значение, конечно, не имеет значения?

В противном случае переменная инициализация осуществляется следующим образом:

Объявление статической или библиотечной переменной с инициализирующим выражением инициализируется лениво ().

Ленивая семантика дается потому, что мы не хотим языка. где имеет тенденцию определять дорогостоящие вычисления инициализации,

длительное время запуска приложений. Это особенно важно для Дарта, который должен поддерживать кодирование клиентских приложений.?

Инициализация переменной экземпляра без инициализации выражения
Происходит во время выполнения конструктором () ?

Инициализация переменной экземпляра с инициализирующим выражением идет следующим образом: оценивается на предмет и переменной Связано с .

Он указывается в другом месте, когда происходит эта инициализация.
и в какой среде (р. ??), , ?

Если инициализирующее выражение бросает тогда доступ к неинициализированной переменной предотвращен; Потому что создание экземпляра Это привело к тому, что инициализация бросок.?

Ошибка динамического типа, если динамический тип не является подтип фактического типа переменной ().

Оценка неявных переменных Геттеров

Мы вводим два конкретных типа геттеров, чтобы указать Поведение различных типов переменных без дублирования спецификации их общего поведения.

А. <позднеинициализированный Getter! (с поздней инициализацией) является геттером который неявно индуцируется нелокальной переменной имеет начальное выражение . Ниже описано, какие объявления вызывают поздний инициализ Геттера.? Призыв идет следующим образом:

Если переменная Не был привязан к объекту тогда оценивается на предмет . Если Теперь он привязан к объекту и является окончательным, Возникает динамическая ошибка. В В противном случае Обязателен для , и оценки Полное возвращение . Если оценка бросок тогда Обращение к завершает бросание одного и того же предмета и следа стека, и не изменяет связывания .

Призыв В ситуации, когда был привязан к объекту Завершается немедленно, возвращаясь .

Рассмотрим нелокальную переменную декларацию формы , чей неявно индуцированный геттер запоздало инициализируется. Возможно, удивительно, Если переменная был привязан к объекту Когда впервые вспыхивает его геттер, Они никогда не будут казнены. Другими словами, выражение инициализации может быть предупредительным. по заданию.?

Также обратите внимание, что инициализирующее выражение может иметь побочные эффекты. Это важно при инициализации. Например: ?

В этом примере призывать неявно индуцированный геттер по имени , и переменной На данный момент он не связан. Таким образом, оценка инициализации выражения продолжается. Это приводит к переключению на , Это приводит к повторной оценке .Это вызывает геттер Чтобы вновь призвать, И все же верно, что переменная не была связана, Таким образом, выражение инициализации оценивается

повторно. Это переключает на , что вызывает оценку , который вызывает неявно индуцированный установщик, названный , и последний вызов геттера Возвращение 10. Это означает, что Оценить 11, Затем переменная привязывается к 11. Наконец, вызов геттера i в main Возвращение 11.?

Это изменение от семантики старых версий Дарта: Выбрасывание исключения во время оценки инициализатора больше не устанавливает переменная на , и считывание переменной во время оценки инициализатора больше не вызывает динамических ошибок. ?

А. поздноинициализированный getter!late-uninitialized Это геттер (P000249). который неявно индуцируется нелокальной переменной Это не имеет начального выражения. Опять же, только некоторые нелокальные переменные без инициализации выражения индуцировать поздно-неинициализированный геттер, как указано ниже.? Призыв идет следующим образом: Если переменная Он не привязан к объекту, поэтому возникает динамическая ошибка. Если Привязан к объекту затем Ссылка на Завершается сразу, возвращаясь .

Объявление нелокальной переменной, называемой . Для краткости, мы будем ссылаться на него ниже как пункт переменная или , или просто как или . Исполнение неявно индуцированного геттера идет следующим образом:

Переменная непоздней инстанции Если объявляет переменную экземпляра, не имеющую модификатора , затем вызов неявного геттера оценивать на Объект, который Это неизбежно.

Невозможно вызвать на объекте геттер экземпляра до того, как объект был инициализирован. Следовательно, переменная непоздней инстанции всегда связана когда используется кеттер.?

Поздняя, инициализированная переменная экземпляра Если объявляет переменную экземпляра который имеет модификатор и начальное выражение, Неявно индуцированный геттер является поздноинициализированный геттер. Это определяет семантику призыва.

Поздняя, неинициализированная переменная экземпляра Если объявляет переменную экземпляра который имеет модификатор не имеет начального выражения, Неявно индуцированный геттер является поздно-неинициализированный геттер. Это определяет семантику призыва.

В этом случае это возможно для быть несвязанным, Но есть также несколько способов связать Для объекта: Конструктор может иметь инициализацию () или запись в списке инициализаторов () Это позволит инициализировать , и неявно индуцированный сеттер по имени мог бы Завершено и выполнено нормально.?

Статическая или библиотечная переменная Если объявляет статическую или библиотечную переменную, Неявно индуцированный геттер выполняется следующим образом:

Непостоянная переменная с инициализатором. В случае, когда имеет инициализирующее выражение и не является постоянным;

Неявно индуцированный геттер Это позднеинициализированный геттер. Это определяет семантику призыва. Обратите внимание, что эти статические или библиотечные переменные могут быть поздней инициализации в том смысле, что они не Модификатор .? Постоянная переменная. Если объявляет постоянную переменную с инициализацией выражения, Результатом выполнения неявно индуцированного геттера является значение постоянного выражения. Обратите внимание, что постоянное выражение не может зависеть от самого себя. Так что циклических ссылок быть не может.? Переменная без инициализатора. Если Объявляет переменную без инициализации выражения и не имеет модификатора, а вызов неявно индуцированного геттера оценивать на Объект, который Это неизбежно.

В этом случае переменная всегда связана с объектом. Это может быть нулевой объект. что является начальным значением некоторых переменных объявлений В данном случае.?

Если объявляет переменную без инициализации выражения и имеет модификатор, неявно индуцированный геттер является позднеинициализированным геттером. Это определяет семантику призыва.

Уменьшите белое пространство после подробного списка: Это всего лишь конечный символ. □

Функции.

Функции абстрагируются над выполняемыми действиями.

<функциональная подпись> ::=

```
<type>? <identifier> <formalParameterPart>
<formalParameterPart> ::= <typeParameters>? <formalParameterList>
<functionBody> ::= ? <=> <выражение> ” (async ’*’? | sync ’*’)) <блокировать>
<block> ::= ” <statements> ” ”
```

Функции могут быть введены посредством деклараций функций.

() , Методические декларации (, Геттерские декларации (), устанавливающие декларации (), и конструкторские декларации (); Они могут быть введены буквальными функциями ().

Функция asynchronousfunction!asynchronous если его тело обозначено или Модификатор. В противном случае функция синхронная!синхронная. Функция - это генератор!генератор если его тело обозначено или Модификатор. Более подробная информация об этих концепциях приводится ниже.

Является ли функция синхронной или асинхронной, является ортогональной к Является ли он генератором или нет. Генераторные функции — это сахар для функций которые производятся систематически, при помощи ленивой функции, которая генерирует отдельные элементы коллекции. Дарт обеспечивает такой сахар в обоих синхронных случаях, Когда человек возвращается, и в асинхронном случае, когда один возвращает поток. Dart также позволяет выполнять синхронные и асинхронные функции. Производит однозначную величину.?